

Lecture Q&A

Interview Practice

The questions interviewers ask about what you have studied so far: embeddings, vector search (IVF, KD-tree, HNSW, PQ), RAG, advanced RAG, graph databases and LangGraph. Each answer is short enough to say out loud in under a minute, and each question points to the page of your **GenAI, RAG & Agents** notes that explains it in full.

resume.mjs

```
// Part 2, Q9: the trap question
const config = { configurable: { thread_id: "pdf-001" } };

await app.invoke(null, config); // resume where it stopped
await app.invoke({}, config);  // a new run from START
```

Embeddings

Cosine vs Euclidean

Brute Force & ANN

Recall@k

IVF

KD-tree

HNSW

Product Quantization

FAISS vs Vector DB

RAG

Chunking

Query Rewriting

Multi-hop & Agentic RAG

Hybrid Search

Reranking

Neo4j & Cypher

LangGraph

Reducers

Checkpointers

Idempotency

How to Use This Q&A

These are the interview questions and answers from our study sessions, for the STRIKE GenAI lectures you have finished. Each answer is short enough to say in under a minute: interviewers want the idea, one example and the trade-off, not a lecture.

- **Part 1, Lectures 8–16:** embeddings, vector search (brute force, IVF, KD-tree, HNSW, PQ), RAG, advanced RAG and graph databases. **31 questions.**
- **Part 2, Lectures 20–21:** LangGraph: state, nodes, edges, reducers, checkpoints, resuming after a crash, and the three pipeline designs. **21 questions.**

✓ How to practise

1. Cover the answer and say your own answer **out loud** in under 45 seconds.
2. Read the answer here and notice what you missed.
3. Do about **10 questions a day**, and mark the ones where you get stuck.
4. The next morning, read the notes pages for the marked ones, then try them again. The tag next to each question, like [Notes p. 77](#), is the page in **GenAI-RAG-Agents-Explained-Simply.pdf** that explains it in full.

📌 Asked most often: start with these

- **Part 1:** Q5, Q6, Q9, Q11, Q15, Q16, Q17, Q21 and Q27.
- **Part 2:** Q6–Q11 and Q13.

Contents

PART 1	Embeddings, Vector Search, RAG and Graph Databases	3
	A. Embeddings: the base for everything	3
	B. Vector databases and search algorithms	3
	C. RAG	5
	D. Advanced RAG (Lecture 14)	6
	E. Graph databases	7
	F. Linking it to your project	8
PART 2	LangGraph: Workflows as State Machines	9
	A. Core concepts	9
	B. Saving progress and resuming	10
	C. The PDF pipeline and Lecture 21's three designs	11
	D. Other features	11
	E. Your project, StanceScope	12

PART 1 Lectures 8–16 · 31 questions

Q&A

Embeddings, Vector Search, RAG and Graph Databases

IN THIS PART

- A. Embeddings (Q1–Q3)
- B. Vector databases and search algorithms (Q4–Q14)
- C. RAG (Q15–Q19)
- D. Advanced RAG, Lecture 14 (Q20–Q24)
- E. Graph databases (Q25–Q29)
- F. Linking it to your project (Q30–Q31)

A. Embeddings: the base for everything

Q1 What is an embedding?

Notes p. 66–68

A list of numbers that represents the meaning of a text: similar meanings give nearby vectors. It comes from a trained model, and the dimensions aren't human-readable. Documents and questions must use the **same model**, because vectors from different models can't be compared.

Q2 Cosine similarity or Euclidean distance: which for text, and why?

Notes p. 68

Cosine, because it measures the **angle** (the direction of meaning) and ignores length. A short text and a long article on the same topic point the same way but have different lengths, so Euclidean would wrongly call them far apart.

Q3 What does normalising do?

Notes p. 70

It scales every vector to length 1. Then the dot product equals cosine similarity, and Euclidean distance gives the same ranking as cosine. Vector databases like it because a dot product is the fastest calculation. PaperRAG normalises for exactly this reason.

B. Vector databases and search algorithms

Q4 Why can't a normal database do similarity search?

Notes p. 74

Normal indexes (B-trees, hashes) need an order or an exact key. Vectors have no natural order, and "find the 10 closest" isn't an exact match, so we need special nearest-neighbour structures.

Q5 What is brute-force (exact) search, and when is it the right choice?

Notes p. 74

Compare the query with every stored vector and keep the top k . The cost is $N \times d$ per query: 1 million vectors $\times$ 768 dimensions = 768 million operations. It's slow but **perfect**. It's the right choice for small data, which is why PaperRAG uses FAISS `IndexFlatIP`.

Q6 What is approximate search (ANN), and what is recall@k?

Notes p. 75

ANN gives up a tiny bit of accuracy for a huge speed-up, often 100 $\times$ or more. Recall@ k measures what you lost: of the true top- k , how many did the index return? Recall@10 = 0.95 means 95% of the real top 10 were found.

Q7 Explain IVF and its four drawbacks.

Notes p. 75–76

Indexing: k-means finds, say, 1,000 cluster centres, and each vector goes into its nearest centre's list.

Searching: compare the query with the 1,000 centres, then search inside the closest cluster. That's about 2,000 comparisons instead of 1,000,000. The four drawbacks:

1. **Training cost:** k-means over all the data.
2. **Border problem:** the true neighbour may sit just over the edge, in another cluster. The fix is searching more clusters (n_{probe}), which is slower.
3. **Changing data:** clusters go stale as new data arrives.
4. **Choosing K:** the number of clusters has to be tuned.

Q8 Why don't we use KD-trees for text embeddings?

Notes p. 76–77

A KD-tree splits space in halves, alternating dimensions, and backtracks across a split line when a closer point might be on the other side. In 2–3 dimensions it skips most branches. In 768 dimensions the search crosses almost every split line, so it backtracks everywhere and becomes brute force. That's the **curse of dimensionality**.

Q9 Explain HNSW.

Notes p. 77

- **Structure:** layer 0 holds every vector, each linked to about M (e.g. 16) of its nearest neighbours. Upper layers hold fewer and fewer nodes, giving long-distance "highway" links, like a skip list.
- **Search:** start at the top layer, hop greedily to whichever neighbour is closer, drop a layer and repeat. At layer 0, run a beam search that keeps the best **efSearch** candidates.
- **Analogy:** a fast train to the right area, then a slow train for the last stops.

Q10 HNSW's settings, advantages and disadvantages?

Notes p. 77–78

- **Settings:** M (links per node: better accuracy, more memory), **efConstruction** (build quality), **efSearch** (accuracy vs speed per query).
- **Advantages:** no training, no border problem, cheap inserts, the best speed/accuracy balance.
- **Disadvantages:** the most memory (full vectors plus links), awkward deletes, and it's still approximate.

Q11 Explain product quantization (PQ).

Notes p. 78–79

- **The problem:** one 1536-dimension vector in 32-bit floats takes 6,144 bytes, so a billion vectors need 6.1 TB of RAM.
- **Compression:** split each vector into chunks, learn a 256-entry codebook per chunk position, and store each chunk as a 1-byte id. A 16-number vector becomes 4 bytes instead of 64.
- **Search:** compute 4 small lookup tables once per query. The distance to any stored vector is then 4 table lookups added together.
- **Cost:** some accuracy is lost, so you **re-rank** the top candidates with their full vectors. **IVF-PQ** is the standard choice at billion scale.

Q12 Which search method would you pick, and when?

Notes p. 79–81

- **Small data:** exact flat search.
- **Best speed and accuracy, and memory is fine:** HNSW.
- **Billions of vectors with tight memory:** IVF-PQ with re-ranking.
- **Already on Postgres:** pgvector.
- **No servers to run:** Pinecone.

Q13 What does a vector database store, and why does metadata matter?

Notes p. 80

Each record has an **id**, the **vector** and **metadata** (chunk text, file, page, user id, date). Metadata enables filtering, like “only this user’s documents”, which is essential for security in multi-user apps.

Q14 FAISS vs a vector database?

Notes p. 80

FAISS is a library that lives in your program’s memory and stores only vectors; you save and load it yourself. That’s why PaperRAG keeps the chunk text in a separate `chunks.jsonl`. A vector database is a server: metadata, filtering, many users, persistence.

C. RAG

Q15 What is RAG, and why use it instead of fine-tuning?

Notes p. 84–86

Retrieve the relevant chunks from your data, put them in the prompt, and let the LLM answer from them: an open-book exam. It fixes the knowledge cutoff, gives access to private data, reduces hallucinations and enables citations. You update it by re-indexing, not retraining. Fine-tuning is for changing **behaviour or style**, not for adding facts.

Q16 Walk me through a RAG pipeline end to end.

Notes p. 85

- **Indexing:** load → chunk → embed → store (vector + text + metadata).
- **Query:** embed the question with the **same model** → search the top-k → build the prompt (rules + chunks + question) → the LLM answers.

Q17 How do you choose chunk size and overlap?

Notes p. 86–87

The course uses 1,000 characters with 200 overlap. Too small loses context; too big mixes topics and blurs the embedding. Overlap keeps a sentence that falls on a boundary whole in at least one chunk. RecursiveCharacterTextSplitter cuts at paragraphs first, then lines, then words. Final values come from testing on an evaluation set.

Q18 How do you reduce hallucinations in RAG?

Notes p. 90

- Tell the model to answer **only** from the context, and give it an escape sentence ("I don't have enough information").
- Use a low temperature.
- Use a similarity threshold, so weak retrievals refuse before the LLM is called (PaperRAG's first guard).
- Require citations.

Q19 What can't basic RAG do?

Notes p. 91

- Handle follow-up questions.
- Answer multi-hop questions.
- Match exact terms, like error codes or names.
- Refuse when retrieval is weak (no cut-off), or cite its sources.
- Measure itself (no evaluation).

These are exactly what advanced RAG fixes.

D. Advanced RAG (Lecture 14)

Q20 How do you handle follow-up questions like "Explain it in detail"?

Notes p. 94–95

Before retrieving, ask the LLM to rewrite the last question into a **standalone** question using the chat history. After "What is Node.js?", the follow-up "Explain it in detail" becomes "Explain Node.js in detail". Then embed and search with that rewritten question.

Q21 What are multi-hop questions, and why does normal RAG fail on them?

Notes p. 95

Questions that need a **chain** of facts. Example: "Is Vedant's account a fraud?" → Vedant sent money to Priya → Priya's phone → used by a company → that company is flagged. Similarity search only finds chunks close to the question, and the later links aren't similar to it at all.

Q22 Agentic RAG vs Graph RAG?

Notes p. 95–96

- **Agentic RAG:** an agent with a search tool retrieves several times, deciding what to look up next. Flexible, but several LLM calls per question, slower and more expensive, and it needs a step limit.
- **Graph RAG:** stores facts as entities and relationships, so following links is a cheap database query. Best when the data is about relationships.

Q23 What is hybrid search? *extra reading, not in the lecture*

Notes p. 96–97

Run keyword search (BM25, good at exact names and codes) and vector search (good at meaning), then merge the two ranked lists with Reciprocal Rank Fusion: $\text{score} = \sum 1/(60 + \text{rank})$.

Q24 What is reranking? *extra reading, not in the lecture*

Notes p. 97

Retrieve wide (say the top 50) with fast vector search, then score each (question, chunk) pair with a more accurate cross-encoder and keep the best 5. The rule: retrieve wide, rerank narrow.

E. Graph databases

Q25 When would you use a graph database instead of SQL?

Notes p. 112

When the questions are about **relationships**: friends of friends, fraud chains, recommendations, “how is A connected to K?”. SQL is better for aggregates over rows (totals, reports) and simple tables.

Q26 Why is SQL slow for “friends of friends of friends”?

Notes p. 108–109

Each hop is a self-join, and every lookup goes through an index over the **whole** friendships table, costing about $\log N$. With 500 connections each: $\log N + 500 \cdot \log N + 2.5 \text{ lakh} \cdot \log N$ lookups. It grows fast, even though you only care about your own neighbourhood.

Q27 What is index-free adjacency?

Notes p. 109

Each node stores direct references to its relationships, so following a link is a pointer jump: about $O(1)$ per hop. The cost depends on how much of the graph you touch, not on the size of the whole database. It's the adjacency list from DSA, made permanent.

Q28 How does Neo4j store this?

Notes p. 110

- **Fixed-size records**: 15 bytes per node, 34 bytes per relationship. Record i lives at $\text{base} + i \times \text{size}$, an $O(1)$ jump like an array.
- **Linked relationships**: a node points to its first relationship, and each relationship points to the next one for both of its nodes, forming linked lists.
- **The starting node** still needs an index (hash or B+ tree).

Q29 Explain the graph data model and Cypher basics.

Notes p. 110–111

- **Nodes** are things, and their **labels** are like classes.
- **Relationships** are typed, directed facts, like `DIRECTED` or `ACTED_IN`.
- **Properties** hold the data.
- **Cypher** follows `MATCH` (a pattern) $\rightarrow$ `WHERE` (filter) $\rightarrow$ `RETURN`. **MERGE** avoids duplicates, and values always go in as `$parameters` to prevent injection.

F. Linking it to your project

Q30 Why does PaperRAG use exact search and not HNSW?

Notes p. 164

A few hundred papers is only tens of thousands of vectors, which exact search handles in milliseconds with perfect recall and no tuning. HNSW would add approximation and settings to solve a speed problem I don't have. At millions of chunks I'd switch, and measure its recall against exact search.

Q31 What would change if PaperRAG had 1 million papers?

Notes p. 167

About 30–50 million chunks. I'd move to HNSW or IVF-PQ in a vector database, measure recall against exact search on a sample, embed offline in batches, keep metadata in a database, add metadata filters, and re-tune the refusal threshold.

✓ How to practise Part 1

Answer in batches of 10 a day, out loud and without looking, and mark the ones where you get stuck. The ones asked most often are **Q5, Q6, Q9, Q11, Q15, Q16, Q17, Q21 and Q27**.

PART 2 Lectures 20–21 · 21 questions

Q&A

LangGraph: Workflows as State Machines

IN THIS PART

- A. Core concepts (Q1–Q5)
- B. Saving progress and resuming (Q6–Q11)
- C. The PDF pipeline and Lecture 21's three designs (Q12–Q16)
- D. Other features (Q17–Q19)
- E. Your project, StanceScope (Q20–Q21)

A. Core concepts

Q1 What is LangGraph, and why not just write a loop?

Notes p. 133–134

LangGraph builds AI workflows as a **state machine**: state, nodes and edges, plus a runtime that **saves progress after every step**. A plain loop keeps its progress only in memory: if it crashes at chunk 237 of 1,000 you start again, and you can't pause for a human or inspect where it stopped. LangGraph gives you resume, loops, branching, parallel steps and human pauses.

Q2 Explain state, nodes and edges.

Notes p. 136

- **State** is the shared data of one run, like `pdfText`, `chunks` and `currentIndex`.
- **Nodes** are functions that read the state and return only the fields they changed.
- **Edges** decide what runs next. They can be fixed, or **conditional**: a router function looks at the state and returns the next node's name or `END`. Loops are built from conditional edges.

Q3 What is a reducer, and when do you need one?

Notes p. 136

A reducer decides how a node's update is merged into a state field. By default the last value wins, which is right for `currentIndex`. For a list like `log` or `messages` you need an **append** reducer; otherwise each update wipes out the earlier entries. Reducers are also needed when two parallel nodes write the same field.

Q4 What happens if two parallel nodes update the same field without a reducer?

Notes p. 140

LangGraph raises an error, because a plain field can only take one value per step. In LangGraph JS it is an `InvalidUpdateError`: "LastValue can only receive one value per step" (I tested it). You add a reducer, such as `append` or `merge`, so both updates are combined.

Q5 LangGraph vs LangChain?

Notes p. 141

LangChain provides building blocks: prompts, models, parsers, retrievers. LangGraph is the runtime that orchestrates them in stateful, looping workflows with checkpoints. They're used together: a LangGraph node can call a LangChain component.

B. Saving progress and resuming

Q6 What is a checkpointer, and what does it save?

Notes p. 137

After every node it saves (**state, next node**) under a `thread_id`. That's what makes resume, pause and inspection possible. Without a checkpointer, a stopped graph has nothing to resume from.

Q7 What is `thread_id` for?

Notes p. 137

It identifies one run. The same `thread_id` continues that run; a new one starts fresh. In a web app it's usually one conversation, one job or one request. StanceScope uses `run-<id>`.

Q8 How do you resume after a crash, and what re-runs?

Notes p. 137–138

Call `invoke(null, config)` with the same `thread_id`. LangGraph loads the last checkpoint and continues from the saved next node. I tested this: it crashed while storing chunk 2, and on resume only `storeEmbedding` ran again. Loading, chunking and the earlier embeddings were not repeated, and the result was exactly 4 vectors with no duplicates.

Q9 Trap question: why `invoke(null, ...)` and not `invoke({}, ...)` ?

Notes p. 138

`null` means "continue the stopped run". Passing new input, even `{}`, starts a **new** run from START on that thread. I tested both: with `null` only the failed node ran again; with `{}` the first node ran again as well.

Q10 MemorySaver vs SqliteSaver vs Postgres?

Notes p. 137

- **MemorySaver** keeps checkpoints in RAM: good for learning and tests, but lost if the process dies.
- **SqliteSaver** uses a file on disk, for single-machine apps.
- **Postgres or Redis** savers are for production with many servers.

Q11 What is idempotency, and why does LangGraph need it?

Notes p. 139

The checkpoint is written **after** a node finishes. If the process dies after the database write but before the checkpoint, that node runs again on resume. So side effects must be safe to repeat: use an **upsert with a fixed id** (like the chunk index), not an insert with a random id. Something like "send email" must go behind a check or a human approval.

C. The PDF pipeline and Lecture 21's three designs

Q12 Why store `currentIndex` in the state?

Notes p. 135

So progress is saved in every checkpoint. After a crash you resume from index 237, not from 0.

Q13 Explain the three designs, and which you'd pick.

Notes p. 139–140

- **One by one:** the simplest, with the finest resume, but one API call and one database write per chunk, so it's slow.
- **Batches:** embed a batch in parallel with `Promise.all` and save it with one bulk upsert. Much faster, but all chunks stay in memory.
- **Streaming batches:** create only the next batch from a text cursor. Low memory, at the cost of a bit more state to track.

For big files I'd pick streaming batches, with fixed ids so resume never duplicates anything.

Q14 Your loop over 1,000 chunks fails with a "recursion limit" error. Why, and what's the fix?

Notes p. 140–141

Each round of the loop counts as steps, and the default limit is 25 in the JS version. The error says "Recursion limit of 25 reached without hitting a stop condition". Raise `recursionLimit` in the config (the course uses 500), or batch the work so there are fewer rounds.

Q15 How would you process 1 million chunks?

Notes p. 141

- Streaming batches.
- Limited parallelism inside each node, with retries and backoff.
- Bulk upserts with fixed ids.
- A persistent checkpointer, so progress survives crashes.
- A recursion limit sized for the loop.

Q16 What if the embedding API rate-limits you in the middle of a run?

Notes p. 137–138

The node fails and the run stops there, but the checkpoint keeps all finished work. I'd retry with exponential backoff inside the node, or resume later with `invoke(null, config)`. Nothing already embedded is redone.

D. Other features

Q17 How do you show live progress to a user?

Notes p. 140

Streaming. `app.stream(input, { streamMode: "updates" })` sends each node's update as it happens. `"values"` streams the full state, and `"messages"` streams the LLM's tokens.

Q18 How do you debug a run?

Notes p. 140

`getState(config)` shows the current values and the next node, and the state history lets you look at any earlier step ("time travel"). Unit-test each node as a plain function, and add one test that crashes in the middle and resumes.

Q19 When would you NOT use LangGraph?

For one LLM call or a short straight-line pipeline. LangGraph adds structure you don't need there; a plain function is simpler.

E. Your project, StanceScope

Q20 How does StanceScope use LangGraph?

Notes p. 191

Four nodes: retrieve evidence → classify posts → human review → aggregate. It uses a checkpointer (MemorySaver) and `thread_id = run-<id>`, so a run can pause for human review and continue later.

Q21 What would you change in StanceScope, and why?

Notes p. 194

Switch MemorySaver to **SqliteSaver**. I tested a server restart while a run was waiting for review: the resume failed with a `KeyError`, because the checkpoint was only in memory. A persistent checkpointer fixes that.

✓ How to practise Part 2

Answer each one out loud in under 45 seconds without looking. Mark the ones where you get stuck, and re-read their pages the next morning. **Q6–Q11 and Q13** come up most often.